In this module

An overview of the Rust ecosystem and tools used during development

Cargo

Most daily usage of Rust will involve using cargo in one way or another, some of the more common tasks are

- Creating new projects
- Managing dependencies
- Building projects
- Executing the resulting binaries
- Running tests and benchmarks
- Generating and viewing local documentation

Cargo configuration

Cargo is managed through the `Cargo.toml` configuration file. Toml is an easy to read configuration file fairly similar to ini files.

```
[package]
name = "example"
version = "0.1.0"
edition = "2021"

[dependencies]
serde = "1.0"
```

Adding dependencies

You can add dependencies to `Cargo.toml` in multiple ways

Add a line in cargo.toml

```toml
[package]
name = "example"
version = "0.1.0"
edition = "2021"

[dependencies]
serde = "1.0"
itertools = "0.10"
```

Use `cargo add`

```
cargo add itertools
```

Dependencies? Crates!

The crate is the compilation unit for Rust

- Binary crates: result in a compiled binary program that you can execute, binaries have a `main` function that acts as the entrypoint of the program
- Library crates: define functionality that can be used by other crates, they have no specific `main` function

Each crate in Rust has a root file, for binary crates this typically is `main.rs`, but for libraries this typically is `lib.rs`.

Libraries

So our dependencies are library crates, but where can we find them?

- Most crates can be found on crates.io, the central community crate registry
- Other options are:
  - Another directory on your local filesystem
  - From a git repository
  - From an alternative registry

```toml
[package]
name = "example"
version = "0.1.0"
edition = "2021"

[registries]
my-registry = { index = "https://my-intranet:8080/path/to/index" }

[dependencies]
my-lib = { path = "../my-lib" }
my-versioned-lib = { git = "https://github.com/example/example.git", branch = "main" } # or use `tag` or `rev`
my-custom = { version = "1.0", registry = "my-registry" }
serde = "1.0"
```

Finding crates

Selecting reliable dependencies is an important part of modern software development, the Rust community has some tools to help you with that

- Look at the dependencies, dependents, number of downloads etc on crates.io
- Look at what crates are manually curated on lib.rs
- Look at the crate documentation on docs.rs

Documentation

The documentation on docs.rs is generated by the build in rustdoc tool

```
cargo doc --open
```

This command will open a browser with the documentation of not just your own crate but also any dependencies you have.

Features

Many crates will have optional features, that is functionality that is only available when a feature flag is provided. You can provide these feature flags through your `Cargo.toml` file:

```toml
[package]
name = "example"
version = "0.1.0"
edition = "2021"

[dependencies]
tokio = { version = "1.23", features = ["net"] }
```

Some crates will have some features enabled by default, to disable those default features you will have to add this:

```toml
tokio = { version = "1.23", features = ["net"], default_features = false }
```

Testing

Rust has a built-in test runner and you can easily add tests to your program.

```
fn main() {
    println!("Hello, world! fib(6) = {}", fib(6));
}

fn fib(n: u64) -> u64 {
    if n <= 1 {
        n
    } else {
        fib(n - 1) + fib(n - 2)
    }
}

#[test]
fn test_fib() {
    assert_eq!(fib(2), 1);
    assert_eq!(fib(5), 5);
    assert_eq!(fib(10), 55);
}
```

```
cargo test
```

Tests in your documentation

You can even use examples in your documentation as tests

```
/// Calculates fibonacci number n
///
/// # Examples
///
/// ```
/// # use example::fib;
/// assert_eq!(fib(2), 1);
/// assert_eq!(fib(5), 5);
/// assert_eq!(fib(55), 55);
/// ```
pub fn fib(n: u64) -> u64 {
    if n <= 1 {
        n
    } else {
        fib(n - 1) + fib(n - 2)
    }
}
```

```
cargo test --doc
```

Compiling

We have documented and tested our code, but how do we run it?

- Use `cargo run` to run your binary during development, this recompiles only if needed
- Use `cargo build` if you just want to compile
- Use `cargo check` if you want to quickly see if there are any issues without generating the resulting binary

Dev/debug vs Release

By default when you run or build Rust will generate a **dev** (often called debug) build

- Add the `--release` flag to create a release build
- Release builds optimize your binary such that it runs faster, but it takes longer to compile
- In dev/debug mode, overflows of integers will error, but they will not in release
- Release builds no longer contain debug information, it won't be as easy to run a debugger on them

Profiles

We can create custom profiles in our `Cargo.toml` that provide alternatives to debug and release compilation

```
[profile.custom]
opt-level = 1 # 0, 1, 2, 3, "s", "z"
debug = 0 # 0/false, 1, 2/true
split-debuginfo = "off" # "off", "packed", "unpacked"
strip = "none" # "none", "debuginfo", "symbols"
debug-assertions = true
overflow-checks = true
lto = true # false, true/"fat", "thin", "off"
panic = "unwind" # "unwind", "abort"
incremental = true
codegen-units = 256 # > 0
```

```
cargo build --profile custom
```

Compiler versions

Most of the time you should be using the latest stable version of the Rust compiler, but there are other options available

- **Stable**: the stable version of Rust has a strong guarantee of backwards compatibility, but only the latest stable release will get security fixes
- **Beta**: the next stable version, use this if you want to see if your code still works on a new Rust or if you want to help testing new Rust releases
- **Nightly**: the unstable Rust compiled each night from the most current code. Nightly does break every once in a while, but it has one additional feature: we can use unstable features.

Unstable features

Some of the most bleeding edge features of the Rust compiler itself are hidden behind feature flags. These flags only work on nightly. The features behind them have no stability guarantee and might change at any time.

Generally you will want to avoid the unstable features except for research of or development on the Rust compiler. Some popular third party tooling does require the use of the nightly compiler, but their code is written by people intimately familiar with the compiler code.

Language editions

Since releasing 1.0, Rust has guaranteed backwards compatibility for every compiler released since.

- Sometimes it makes sense to make a few small adjustments to the language
- Every few years Rust has the ability to release a new language edition
- You must opt in to newer Rust editions via command line flags or `Cargo.toml`
- Editions are set per crate
- Crates you depend on do not need to be on the same edition as you are
- All editions of Rust map to the same internal compiler structures
- Generally newer editions are easier to use and should be preferred

```toml
[package]
name = "example"
version = "0.1.0"
edition = "2021" # currently available are 2015, 2018 and 2021
```

Rustup

To manage compiler updates you can use rustup

- Rustup adds an additional feature to cargo: `` `cargo +nightly` `` that allows you to run a single command on another compiler version.
- `` `rustup update` `` updates your compiler to the latest available version
- Rustup not just manages the compiler itself, it also manages compiler documentation and standard libraries.

Standard libraries

Rust is ready for cross compilation right from the start! The only* thing you need is a standard library for your target platform.

target	notes
`aarch64-unknown-linux-gnu`	ARM64 Linux (kernel 4.1, glibc 2.17+)
`i686-pc-windows-gnu`	32-bit MinGW (Windows 7+)
`i686-pc-windows-msvc`	32-bit MSVC (Windows 7+)
`i686-unknown-linux-gnu`	32-bit Linux (kernel 3.2+, glibc 2.17+)
`x86_64-apple-darwin`	64-bit macOS (10.7+, Lion+)
`x86_64-pc-windows-gnu`	64-bit MinGW (Windows 7+)
`x86_64-pc-windows-msvc`	64-bit MSVC (Windows 7+)

Maintaining code

Having maintainable code is even better than just being able to write it easily. Rust provides a few tools that will help you:

- Format your code following standard practices: `cargo fmt`
- Catch common mistakes and improve your code: `cargo clippy`

More resources

There is a lot of documentation available

- Standard Library: documentation for the standard library
- The Rust Programming Language: a free online book explaining the entire programming language. Print versions are also available.
- Language reference: a reference of all the language features
- Rust by example: quick and simple examples of the Rust language
- Rustonomicon: details about unsafe Rust
- cheats.rs: A quick reference language cheat sheet